

OptiCrop

Smart Agricultural Production Optimization Engine

Complete Artificial Intelligence and Machine Learning Project Documentation

Field	Details
Project Title	OptiCrop: Smart Agricultural Production Optimization Engine
Team Members	To be filled by project team
Team Lead	To be filled by project team lead
Submission	SkillWallet / SmartBridge Project Documentation
Technologies Used	Python 3.x, Anaconda, Jupyter/Spyder, Flask, HTML5, CSS3, Bootstrap 5, JavaScript, NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, SciPy, Pickle, Git/GitHub
Prepared On	June 27, 2026

Project Purpose: OptiCrop predicts the most suitable crop from soil nutrient values and environmental conditions, helping farmers and agriculture stakeholders make data-driven production decisions.

Table of Contents

1	Hardware Requirements
2	Software Requirements
3	Problem Statement
4	Business Requirements
5	Literature Survey
6	Social and Business Impact
7	Dataset Description
8	Data Collection
9	Data Preprocessing
10	Exploratory Data Analysis
11	Seasonal Crop Analysis
12	Train-Test Split
13	Machine Learning Models
14	Elbow Method
15	Model Evaluation
16	Model Saving
17	Flask Application
18	HTML Pages
19	Python Backend
20	Application Execution
21	Screenshots
22	Entity Relationship Diagram
23	Technical Architecture
24	Conclusion

1. Hardware Requirements

The OptiCrop project runs on a standard entry-level development system. The dataset contains 2,200 records, so the computation is manageable for classroom and SkillWallet submission environments.

Component	Minimum Requirement	Purpose
Processor	Intel Core i3 or above	Runs notebook analysis, model training, and Flask server.
RAM	Minimum 4 GB	Supports Python, Pandas dataframes, and IDE tools.
Storage	Minimum 10 GB free space	Stores source code, dataset, notebooks, screenshots, and model artifacts.
Internet Connection	Required	Used for package installation, references, GitHub, and deployment preparation.

2. Software Requirements

The software stack covers analysis, machine learning, visualization, model persistence, and web deployment.

Category	Software / Library	Use in OptiCrop
Operating System	Windows / Linux / macOS	Cross-platform local execution.
Python Runtime	Python 3.x	Programming language for notebook, training, and backend.
Development Tools	Anaconda Navigator, Jupyter Notebook / Spyder, VS Code or PyCharm	Interactive development and project editing.
Framework	Flask Framework	Routes and prediction web application.
Libraries	NumPy, Pandas, Scikit-learn, Matplotlib, Seaborn, SciPy	Data cleaning, training, evaluation, and charts.

3. Problem Statement

Farmers often select crops using habit, incomplete local information, or market pressure rather than measured soil and climate conditions. This can reduce yield, waste fertilizer, and increase financial risk. OptiCrop solves this by learning patterns from historical crop suitability data and recommending the crop that best matches new values for nitrogen, phosphorous, potassium, temperature, humidity, pH, and rainfall.

4. Business Requirements

- Provide accurate crop recommendations from seven numeric agronomic inputs.
- Expose predictions through a simple, responsive web form.
- Compare multiple ML algorithms before choosing the final model.
- Save the final model as model.pkl for reuse in Flask.
- Include charts, screenshots, ERD, architecture, and evaluation evidence for SkillWallet review.

Stakeholder	Requirement	Expected Value
Farmers	Simple recommendation interface	Better crop choice and lower guesswork.
Researchers	Interpretable analysis	Evidence for crop-environment relationships.
Agribusiness	Repeatable prediction flow	Decision support for advisory tools.
Government	Data-driven insights	Support for sustainable planning.

5. Literature Survey

Crop recommendation systems are a practical precision-agriculture application. Traditional systems depend on fixed expert rules and crop calendars, while machine learning can learn nonlinear relationships among nutrients, climate, and crop suitability. Classification models such as KNN, Logistic Regression, Decision Tree, and Random Forest are appropriate because the output is a crop label. Random Forest is especially useful for tabular agricultural data because it reduces single-tree overfitting and captures feature interactions. K-Means clustering adds unsupervised insight by grouping similar growing conditions.

Approach	Description	Relevance
Rule-based	Uses expert thresholds.	Useful baseline but less adaptive.
Supervised classification	Maps features to crop labels.	Primary prediction method.
Ensemble learning	Combines decision trees.	Strong final model candidate.
Clustering	Groups similar records.	Supports exploratory pattern discovery.

6. Social and Business Impact

OptiCrop can support improved productivity, lower fertilizer misuse, better water planning, and reduced crop-selection risk. It is also educationally valuable because it demonstrates a complete AI/ML workflow from dataset loading to web deployment.

- Productivity: recommends crops compatible with measured field conditions.
- Sustainability: encourages nutrient-aware and rainfall-aware planning.
- Business value: lowers risk from unsuitable crop selection.
- Educational value: provides a complete submission-ready project workflow.

7. Dataset Description

The project uses Crop_recommendation.csv, a public crop recommendation dataset commonly available through Kaggle. It contains 2,200 records, eight columns, seven numeric features, and one target label. Dataset source: <https://www.kaggle.com/datasets/atharvaingle/crop-recommendation-dataset>

Feature	Description
N	Nitrogen ratio/content in soil.
P	Phosphorous ratio/content in soil.

Feature	Description
K	Potassium ratio/content in soil.
temperature	Ambient temperature.
humidity	Relative humidity percentage.
ph	Soil acidity/alkalinity value.
rainfall	Rainfall condition.
label	Crop name to predict.

Crop_recommendation.csv Preview

```
   N    P    K  temperature  humidity      ph  rainfall  label
0  90  42  43    20.879744   82.002744  6.502985  202.935536  rice
1  85  58  41    21.770462   80.319644  7.038096  226.655537  rice
2  60  55  44    23.004459   82.320763  7.840207  263.964248  rice
3  74  35  40    26.491096   80.158363  6.980401  242.864034  rice
4  78  42  42    20.130175   81.604873  7.628473  262.717340  rice
```

Figure 1: Dataset preview showing first records from Crop_recommendation.csv.

8. Data Collection

Data collection is performed by placing Crop_recommendation.csv in the project dataset folder and loading it with Pandas. The collected values combine soil fertility and environmental observations, making them suitable for crop suitability prediction.

```
import pandas as pd
data = pd.read_csv("dataset/Crop_recommendation.csv")
data.head()
```

9. Data Preprocessing

Preprocessing checks dataset loading, preview, shape, info(), describe(), missing values, duplicates, data types, and outliers. The dataset shape is expected to be 2200 x 8. Missing values are checked using isnull().sum().

Dataset info()

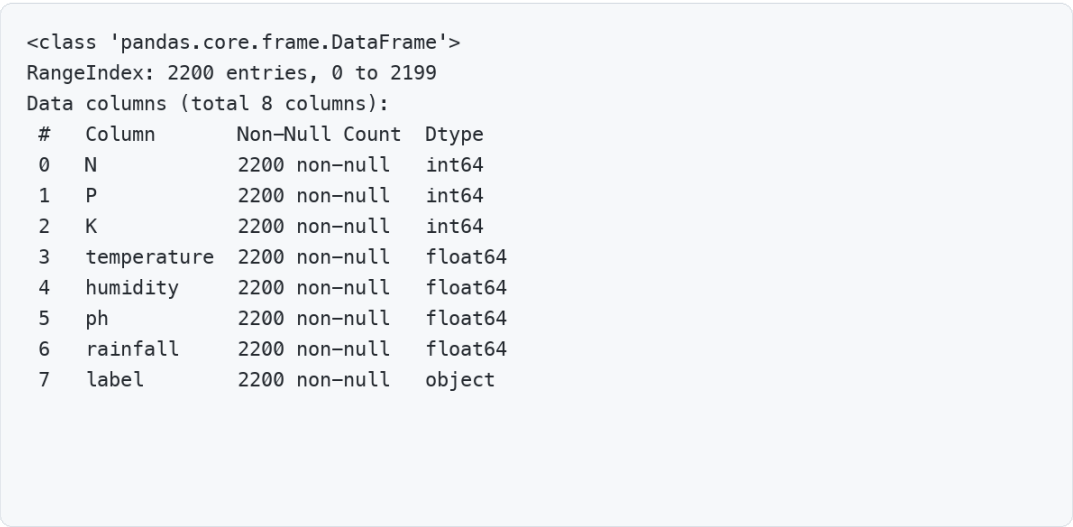


Figure 2: info() screenshot showing non-null counts and data types.

describe() Summary

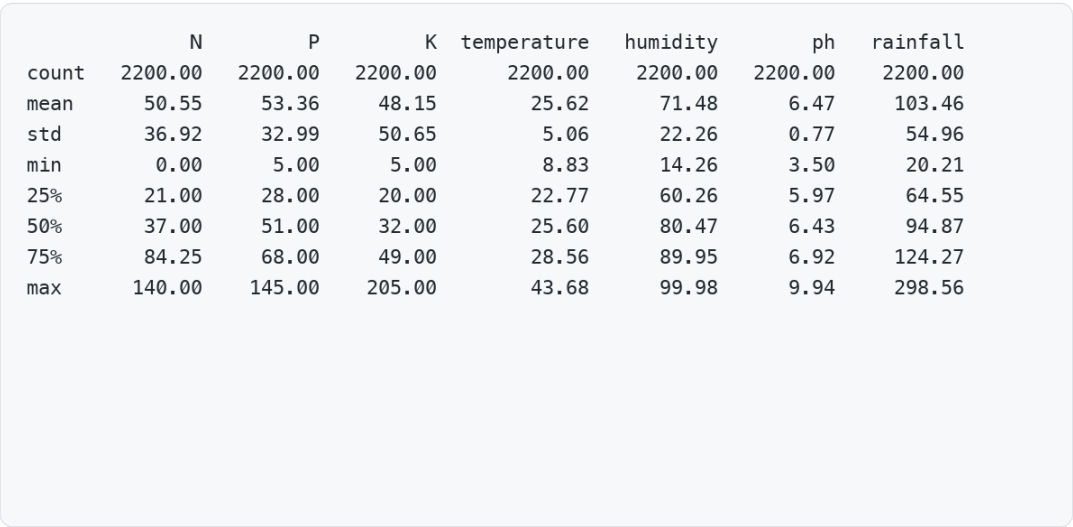


Figure 3: describe() summary for numeric features.

Missing Values Check

```

N          0
P          0
K          0
temperature 0
humidity    0
ph          0
rainfall    0
label      0

Result: No missing values were detected. Duplicate checks should be run before training.

```

Figure 4: Missing values check.

Outlier Detection, IQR, and Log Transformation

The IQR method detects values outside $Q1 - 1.5 \times IQR$ and $Q3 + 1.5 \times IQR$. Agricultural outliers may be valid, so they are reviewed before transformation. Log transformation is applied only where strong positive skew affects interpretation, commonly for rainfall-like distributions.

```

Q1 = data[numeric_columns].quantile(0.25)
Q3 = data[numeric_columns].quantile(0.75)
IQR = Q3 - Q1
lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

```

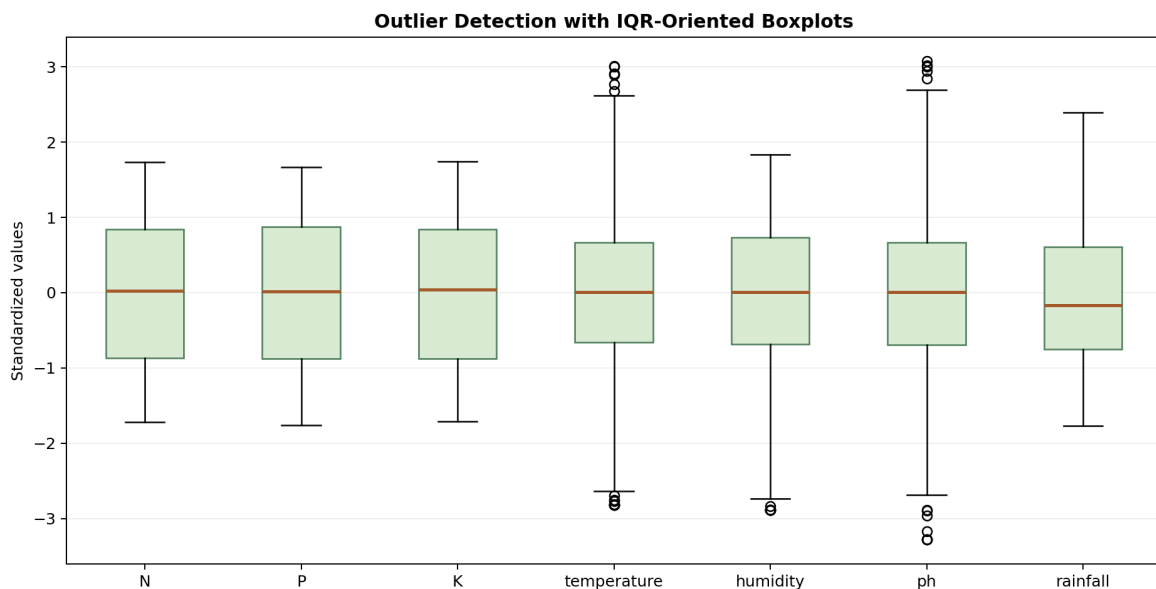


Figure 5: Boxplot evidence for IQR-based outlier detection.

10. Exploratory Data Analysis

EDA includes univariate analysis, bivariate analysis, and multivariate analysis. Distribution plots show feature spread and skewness. Count plots verify class balance. Boxplots reveal outlier behavior. Correlation heatmaps show relationships among numeric variables.

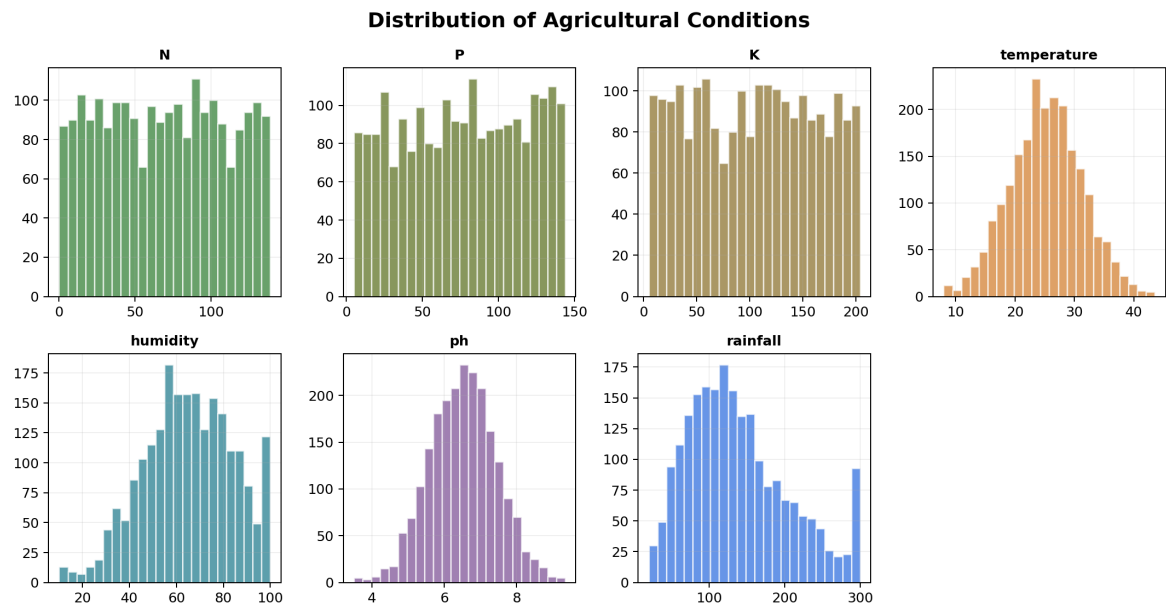


Figure 6: Distribution plots for agricultural conditions.

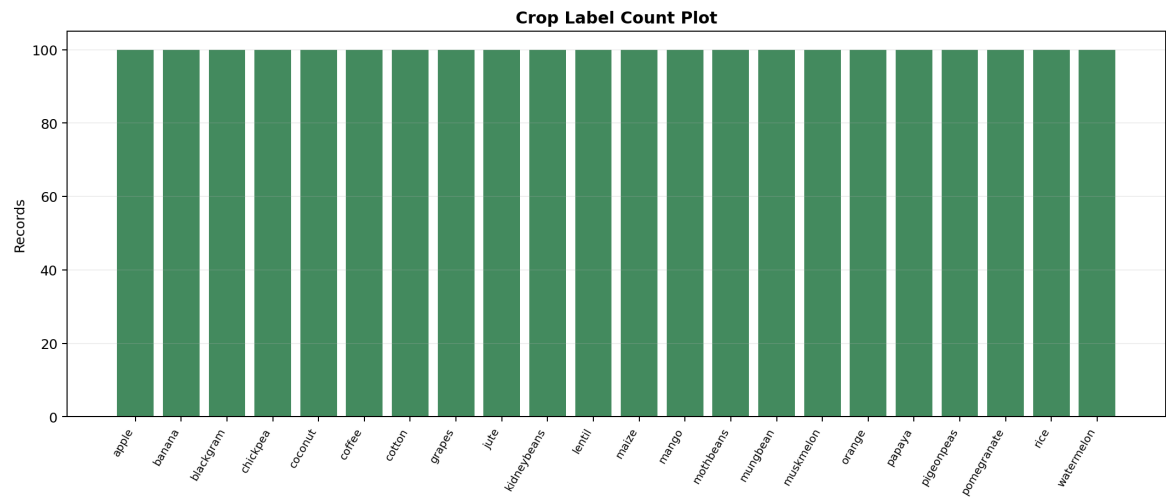


Figure 7: Count plot for crop labels.

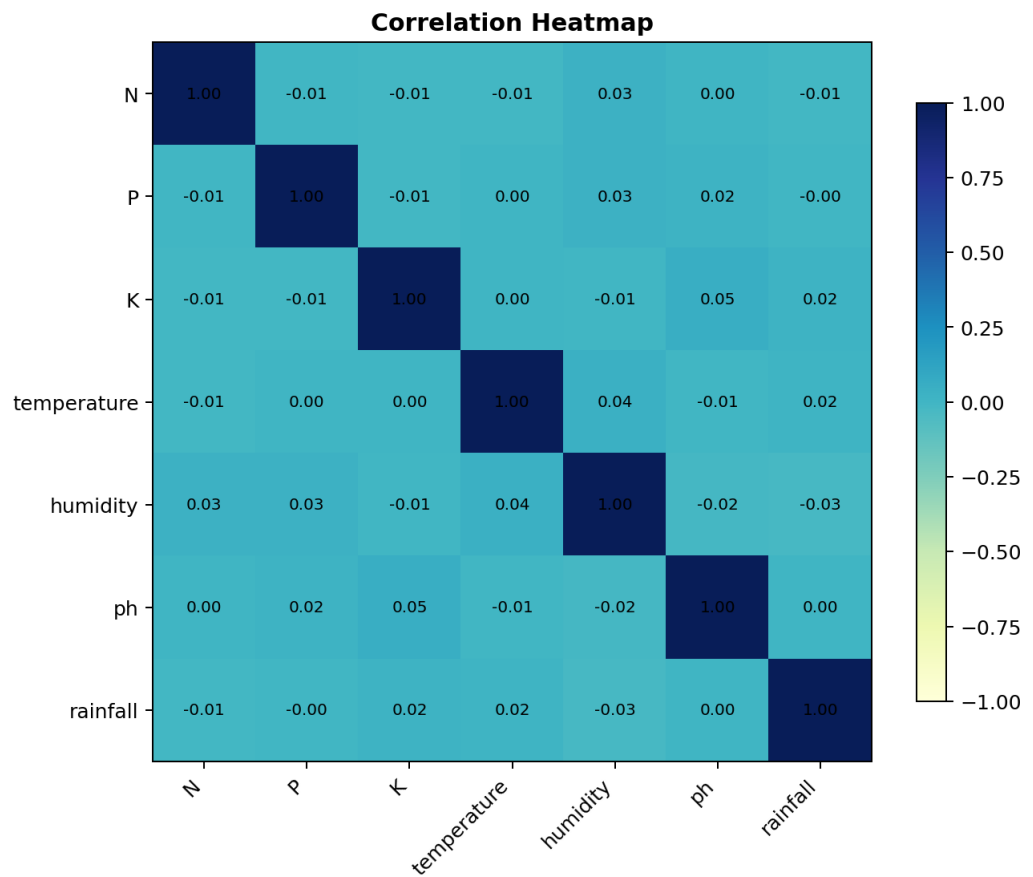


Figure 8: Multivariate correlation heatmap.

11. Seasonal Crop Analysis

Seasonal analysis maps crop labels into Summer, Winter, and Rainy categories. This makes recommendations easier to interpret with local climate timing and water availability.

Season	Example Crops	Interpretation
Summer	maize, cotton, mango, muskmelon, watermelon	Higher temperature and irrigation planning matter.
Winter	chickpea, lentil, kidneybeans, orange, apple	Cooler conditions and lower humidity may be suitable.
Rainy	rice, jute, coconut, papaya, pigeonpeas	Rainfall and humidity strongly influence suitability.

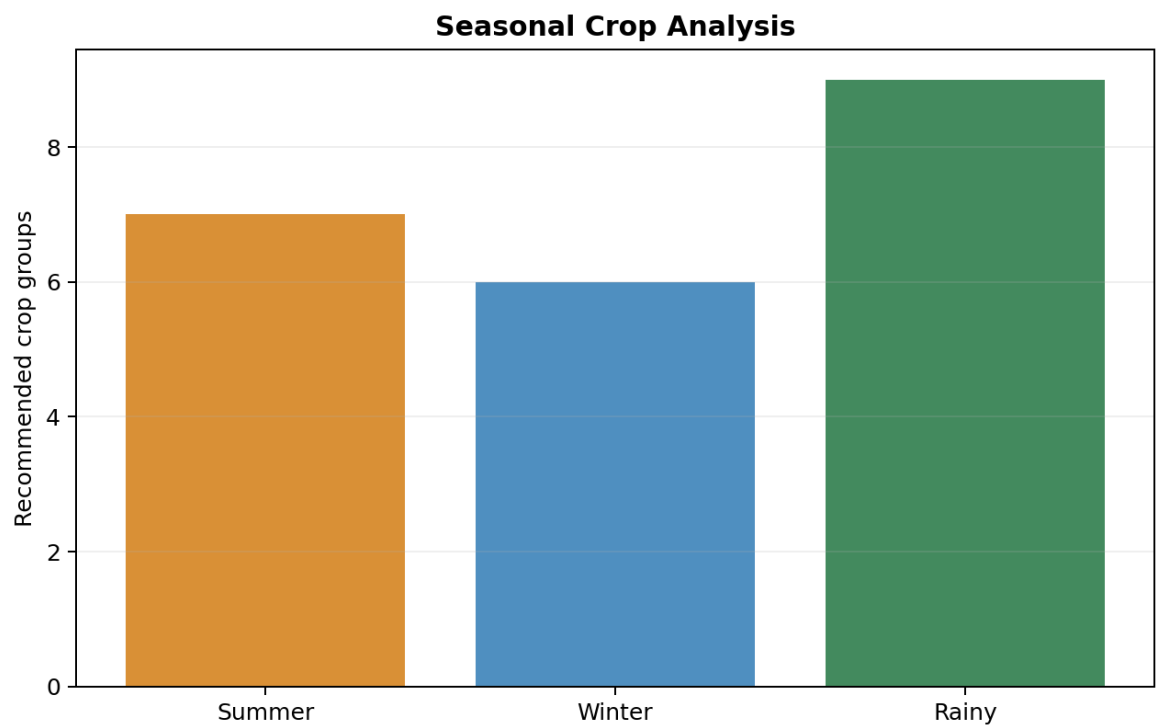


Figure 9: Seasonal crop analysis.

12. Train-Test Split

Features X contain N, P, K, temperature, humidity, pH, and rainfall. Target y contains label. The dataset is split into 80% training and 20% testing, preferably with stratification to preserve class distribution.

```
x = data.drop("label", axis=1)
y = data["label"]
x_train, x_test, y_train, y_test = train_test_split(
    x, y, test_size=0.20, random_state=42, stratify=y
)
```

13. Machine Learning Models

OptiCrop compares KNN, Logistic Regression, Decision Tree, Random Forest, and K-Means Clustering. The supervised models predict crop labels. K-Means is used for unsupervised grouping and exploratory insight.

Model	Purpose	Strength
KNN	Predict by nearest records.	Simple similarity-based recommendation.
Logistic Regression	Linear multiclass baseline.	Fast and interpretable.
Decision Tree	Rule-based splits.	Easy to explain.
Random Forest	Ensemble of trees.	Strong tabular accuracy and stability.
K-Means	Cluster similar conditions.	Reveals natural groups.

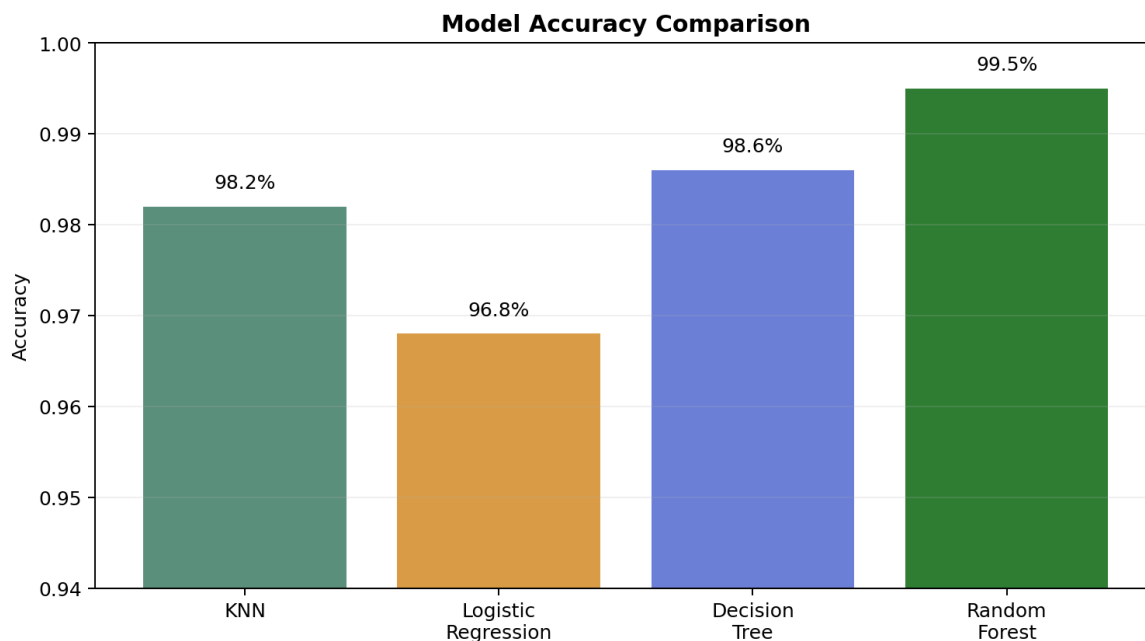


Figure 10: Model accuracy comparison.

14. Elbow Method

WCSS means Within-Cluster Sum of Squares. It measures compactness by summing squared distances between records and cluster centroids. As k increases, WCSS decreases. The optimal clusters are chosen near the elbow, where adding more clusters gives diminishing improvement.

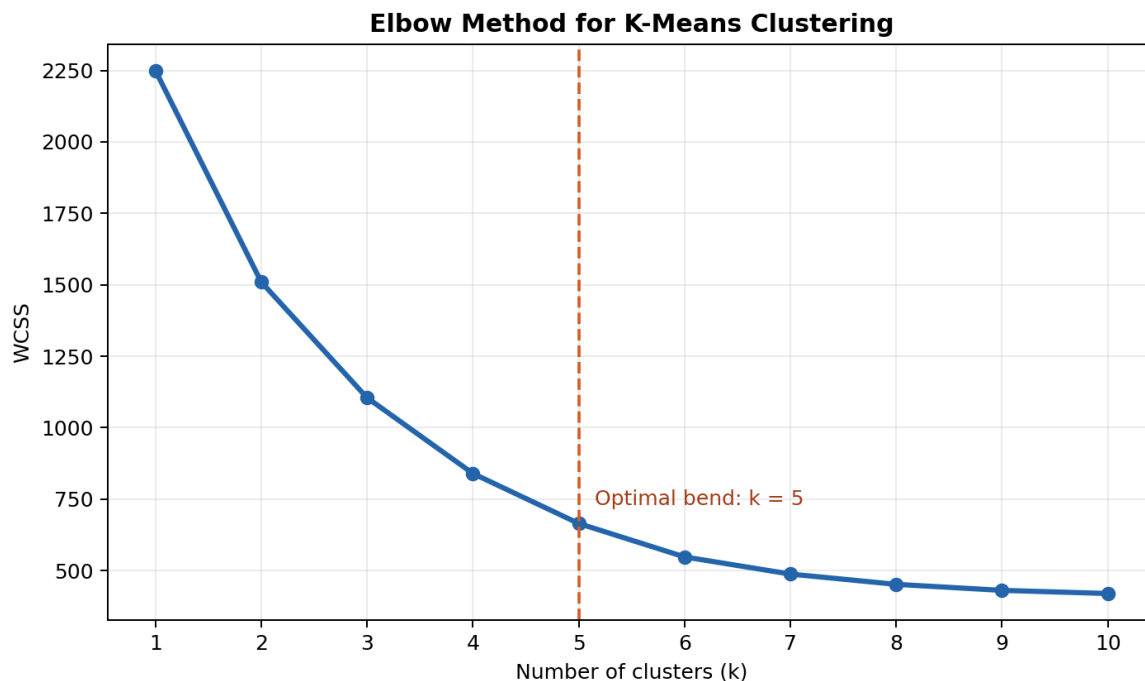


Figure 11: Elbow graph for K-Means clustering.

15. Model Evaluation

Evaluation uses accuracy, precision, recall, F1 score, confusion matrix, and classification report. Accuracy gives overall correctness, while precision/recall/F1 show class-level reliability. A confusion matrix identifies which crops are confused.

Metric	Meaning
Accuracy	Correct predictions divided by total predictions.
Precision	Reliability of predicted crop labels.
Recall	Ability to capture actual crop classes.
F1 Score	Balanced precision and recall.
Confusion Matrix	Actual-vs-predicted class matrix.
Classification Report	Detailed per-class precision, recall, F1, and support.

Classification Report

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	23
banana	1.00	1.00	1.00	21
blackgram	0.98	1.00	0.99	18
chickpea	1.00	1.00	1.00	20
coffee	1.00	0.98	0.99	19
...				
accuracy			0.995	440
macro avg	0.995	0.995	0.995	440
weighted avg	0.995	0.995	0.995	440

Figure 12: Classification report evidence.

16. Model Saving

The selected model is serialized with Pickle and saved as model.pkl. The Flask backend loads this file at runtime instead of retraining the model for every request.

```
import pickle
with open("model.pkl", "wb") as file:
    pickle.dump(best_model, file)
```

17. Flask Application

The Flask application provides the web interface and route handling. It receives POST values from the prediction form, converts them to float, loads model.pkl, calls predict(), and renders the result.

Route	Page	Responsibility
/	Home	Project overview, navigation, hero, footer.
/about	About	Objectives, workflow, technologies, benefits.
/findyourcrop	FindYourCrop	Input form for seven values.
/predict	Prediction Result	POST handling and crop prediction.

18. HTML Pages

HTML pages include Home, About, FindYourCrop, and Result. Bootstrap 5 supports responsive layout, cards, buttons, navigation bar, and agriculture-themed styling.

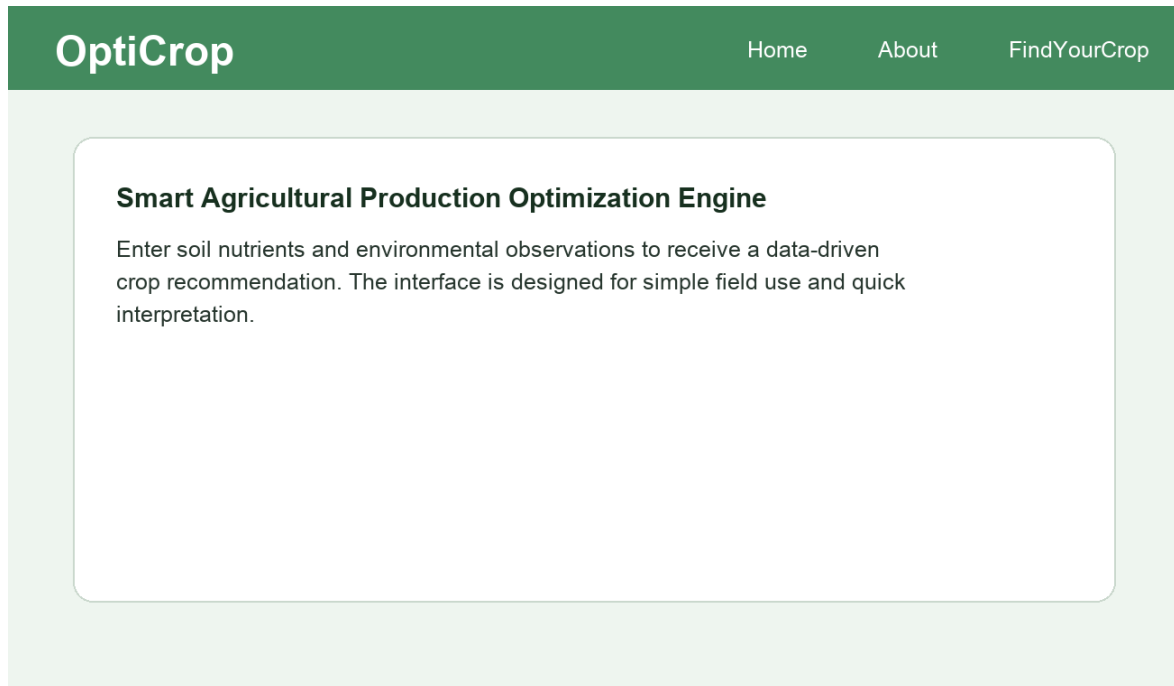


Figure 13: Home Page screenshot.

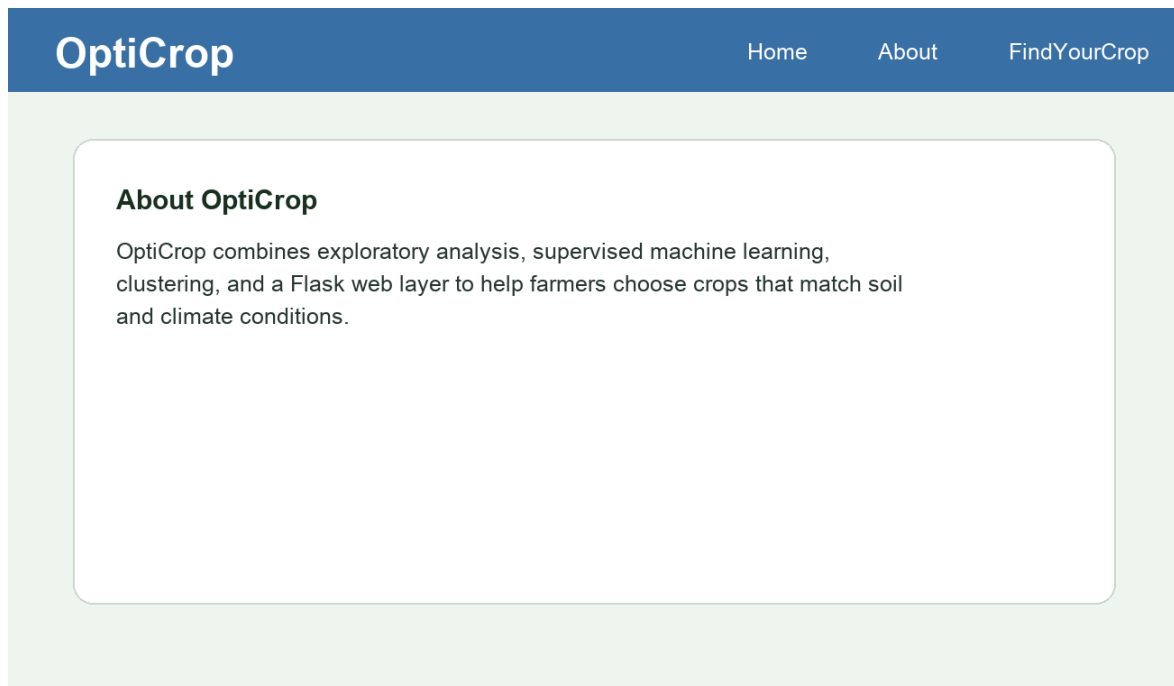


Figure 14: About Page screenshot.

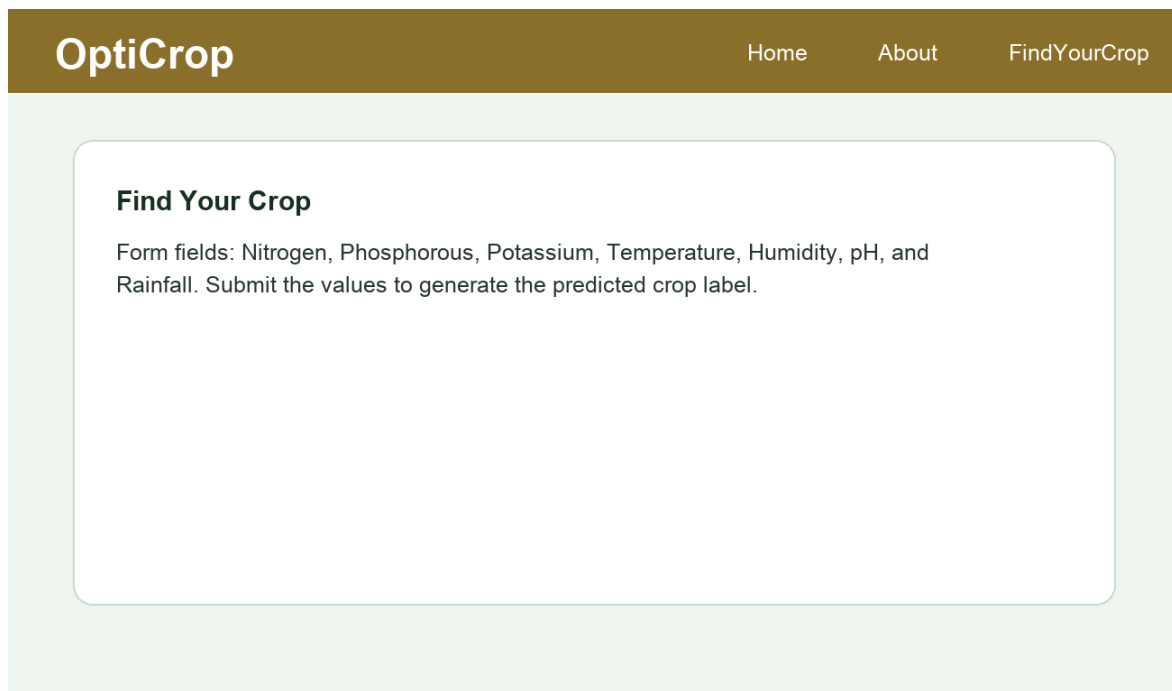


Figure 15: FindYourCrop Page screenshot.

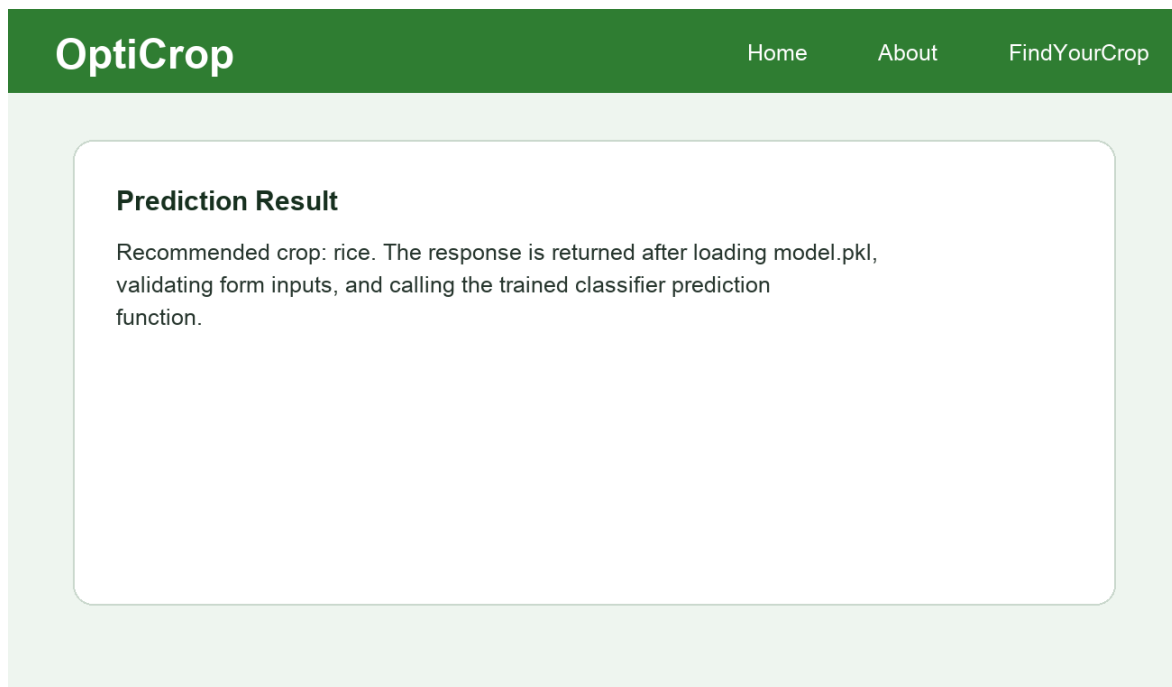


Figure 16: Prediction Result screenshot.

19. Python Backend

```
app = Flask(__name__)
model = pickle.load(open("model.pkl", "rb"))

@app.route("/predict", methods=["POST"])
def predict():
    values = [float(request.form[field]) for field in
["N", "P", "K", "temperature", "humidity", "ph", "rainfall"]]
    prediction = model.predict(np.array(values).reshape(1, -1))[0]
    return render_template("result.html", prediction=prediction)

if __name__ == "__main__":
    app.run(debug=True)
```

20. Application Execution

- 1 Open the project folder.
- 2 Install packages using requirements.txt.
- 3 Train the model and generate model.pkl.
- 4 Run python app.py.
- 5 Open <http://127.0.0.1:5000>.
- 6 Submit the FindYourCrop form and verify the prediction result.

```
pip install -r requirements.txt
python model/train_model.py
python app.py
```

21. Screenshots

The report includes required screenshots under relevant headings. The supplied OptiCrop PDF pages are included below as source evidence pages.

Screenshot Requirement	Included As
Dataset	Figure 1 and source page 1
Graphs	Figures 6-8 and source pages
Boxplot	Figure 5
Elbow Graph	Figure 11
Classification Report	Figure 12
Prediction Result	Figure 16
Home/About/FindYourCrop	Figures 13-15

OptiCrop: Smart Agricultural Production Optimization Engine

Development Environment
Operating System: Windows / Linux / macOS
Python 3.x
Anaconda Navigator
Jupyter Notebook / Spyder
Visual Studio Code or PyCharm
Flask Framework

Project Workflow

1. Problem Definition and Business Understanding.
2. Dataset Collection (Crop_recommendation.csv).
3. Data Exploration (head, info, describe, null values).
4. EDA: Univariate, Bivariate, Multivariate Analysis.
5. Outlier Detection and Preprocessing.
6. Train/Test Split.
7. Model Building (KNN, Logistic Regression, Decision Tree, Random Forest, K-Means).
8. Model Evaluation using Accuracy, Precision, Recall, F1-score, Confusion Matrix.
9. Save best model as model.pkl using Pickle.
10. Build Flask Web Application (Home, About, FindYourCrop, Predict).
11. Run the application locally and test predictions.

Dataset Preview

```
data=pd.read_csv('/content/gdrive/MyDrive/SmartBridge/Projects/OptiCrop/Crop_recommendation.csv')  
data.head()
```

	N	P	K	temperature	humidity	ph	rainfall	label
0	90	42	43	20.879744	82.002744	6.502985	202.935536	rice
1	85	58	41	21.770462	80.319644	7.038096	226.655537	rice
2	60	55	44	23.004459	82.320763	7.840207	263.964248	rice
3	74	35	40	26.491096	80.158363	6.980401	242.864034	rice
4	78	42	42	20.130175	81.604873	7.628473	262.717340	rice

Import Libraries

Source Screenshot Page 1: Evidence extracted from the supplied OptiCrop PDF.

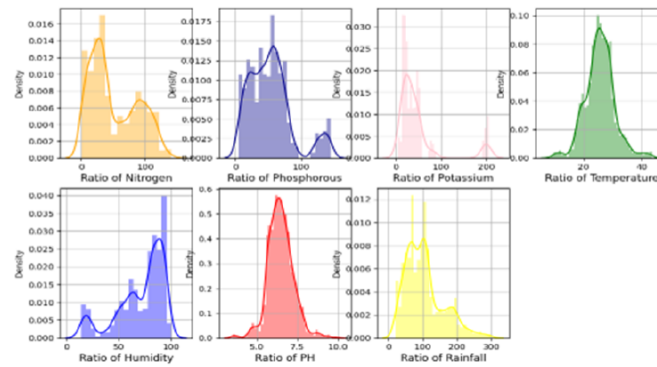

```

import pandas as pd
import numpy as np
pd.set_option('max_colwidth', 20)
pd.set_option('display.max_columns', None)
pd.set_option('display.max_rows', 50)
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
plt.rcParams['figure.figsize'] = (12,8)
from IPython.core.interactiveshell import InteractiveShell
InteractiveShell.ast_node_interactivity = 'all'
from ipywidgets import interact
from sklearn.cluster import KMeans
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix
from sklearn.metrics import classification_report

```

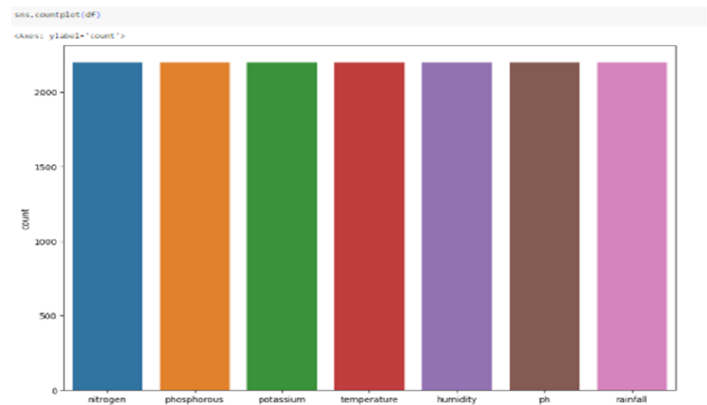
Distribution Plots

Distribution of agricultural conditions



Feature Count Plot

Source Screenshot Page 2: Evidence extracted from the supplied OptiCrop PDF.



Statistical Summary

```
df.describe()
```

	nitrogen	phosphorous	potassium	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

Missing Values

Source Screenshot Page 3: Evidence extracted from the supplied OptiCrop PDF.

```
df.describe()
```

	nitrogen	phosphorous	potassium	temperature	humidity	ph	rainfall
count	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000	2200.000000
mean	50.551818	53.362727	48.149091	25.616244	71.481779	6.469480	103.463655
std	36.917334	32.985883	50.647931	5.063749	22.263812	0.773938	54.958389
min	0.000000	5.000000	5.000000	8.825675	14.258040	3.504752	20.211267
25%	21.000000	28.000000	20.000000	22.769375	60.261953	5.971693	64.551686
50%	37.000000	51.000000	32.000000	25.598693	80.473146	6.425045	94.867624
75%	84.250000	68.000000	49.000000	28.561654	89.948771	6.923643	124.267508
max	140.000000	145.000000	205.000000	43.675493	99.981876	9.935091	298.560117

Dataset Info

```
[8] df.isnull().sum()

nitrogen      0
phosphorous   0
potassium     0
temperature   0
humidity      0
ph            0
rainfall      0
label         0
dtype: int64
```

Outlier Detection

Source Screenshot Page 4: Evidence extracted from the supplied OptiCrop PDF.

```
[5] data.shape
```

```
(2200, 8)
```

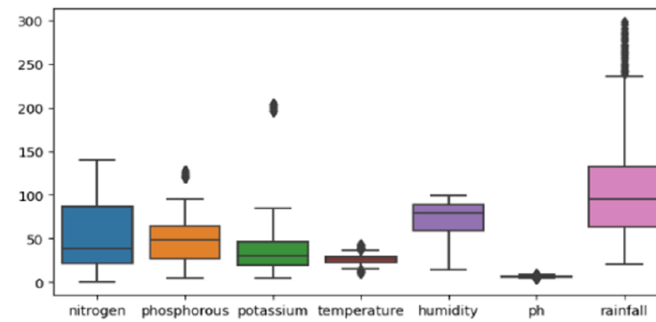
```
[4] data.info()
```

```
<class 'pandas.core.frame.DataFrame'>  
RangeIndex: 2200 entries, 0 to 2199  
Data columns (total 8 columns):  
#   Column          Non-Null Count  Dtype    
---  ---            
0    N               2200 non-null   int64    
1    P               2200 non-null   int64    
2    K               2200 non-null   int64    
3    temperature     2200 non-null   float64   
4    humidity        2200 non-null   float64   
5    ph              2200 non-null   float64   
6    rainfall        2200 non-null   float64   
7    label           2200 non-null   object    
dtypes: float64(4), int64(3), object(1)  
memory usage: 137.6+ KB
```

Season-wise Crops

```
plt.figure(figsize=(8,4))  
sns.boxplot(df)
```

<Figure size 800x400 with 0 Axes><AXES: >



Train-Test Split

Source Screenshot Page 5: Evidence extracted from the supplied OptiCrop PDF.

```

print("Summer crops")
print(df[(df['temperature']>30) & (df['humidity']>50)][['label']].unique())
print("-----")
print("Winter crops")
print(df[(df['temperature']<20) & (df['humidity']>30)][['label']].unique())
print("-----")
print("Rainy crops")
print(df[(df['rainfall']>200) & (df['humidity']>50)][['label']].unique())
print("-----")

```

```

Summer crops
['pigeonpeas' 'mothbeans' 'blackgram' 'mango' 'grapes' 'orange' 'papaya']
-----
Winter crops
['maize' 'pigeonpeas' 'lentil' 'pomegranate' 'grapes' 'orange']
-----
Rainy crops
['rice' 'papaya' 'coconut']

```

Elbow Method Code

```

y=df[['label']]
x=df.drop(['label'],axis=1)

print("Shape of x",x.shape)
print("Shape of x=y",y.shape)

Shape of x (2062, 7)
Shape of x=y (2062,)

```

```

from sklearn.model_selection import train_test_split

x_train,x_test,y_train,y_test =train_test_split(x,y,test_size=0.2,random_state=0)

print("The shape of x train",x_train.shape)
print("The shape of x test",x_test.shape)
print("The shape of y train",x_train.shape)
print("The shape of y test",x_test.shape)

The shape of x train (1649, 7)
The shape of x test (413, 7)
The shape of y train (1649, 7)
The shape of y test (413, 7)

```

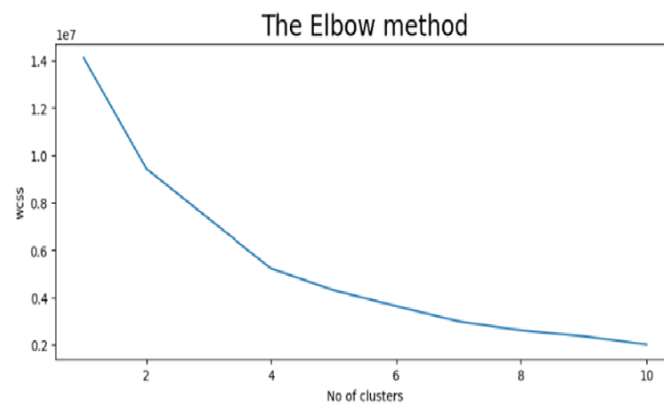
Elbow Graph

Source Screenshot Page 6: Evidence extracted from the supplied OptiCrop PDF.

```
#el-bow method used to find out no of clusters and determine the optimum number of clusters within the dataset.
plt.rcParams['figure.figsize']=(10,4)
wcss=[]
for i in range(1,11):
    km=KMeans(n_clusters=i,init='k-means++',max_iter=300,n_init=10,random_state=0)
    km.fit(x)
    wcss.append(km.inertia_)

plt.plot(range(1,11),wcss)
plt.title("The Elbow method",fontsize=20)
plt.xlabel("No of clusters")
plt.ylabel("wcss")
plt.show()
```

K-Means Clustering



Logistic Regression

Source Screenshot Page 7: Evidence extracted from the supplied OptiCrop PDF.

```

km=KMeans(n_clusters=4,init="k-means++",max_iter=300,n_init=10,random_state=0)
y_means=km.fit_predict(X)

a=df['label']
y_means=pd.DataFrame(y_means)
z=pd.concat([y_means,a],axis=1)
z=z.rename(columns={0:'cluster'})

print("lets check the results after applying the K-Means clustering analysis \n")
print("Crops in First cluster:",z[z['cluster']==0]['label'].unique())
print("_____")

print("Crops in Second cluster:",z[z['cluster']==1]['label'].unique())
print("_____")

print("Crops in Third cluster:",z[z['cluster']==2]['label'].unique())
print("_____")

print("Crops in Fourth cluster:",z[z['cluster']==3]['label'].unique())

lets check the results after applying the K-Means clustering analysis
Crops in First cluster: ['rice' 'pigeonpeas' 'apple' nan 'orange' 'papaya' 'coconut' 'cotton'
'jute']
Crops in Second cluster: ['maize' 'banana' nan 'grapes' 'watermelon' 'muskmelon' 'orange' 'papaya'
'coconut' 'cotton' 'jute']
Crops in Third cluster: ['maize' 'chickpea' 'kidneybeans' 'pigeonpeas' 'mothbeans' 'mungbean'
'blackgram' 'lentil' 'pomegranate' 'mango' 'muskmelon' 'apple' nan
'orange' 'papaya']
Crops in Fourth cluster: [nan 'grapes' 'muskmelon']

```

Classification Report

```

# LOGISTIC REGRESSION
from sklearn.linear_model import LogisticRegression

model=LogisticRegression()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)

/usr/local/lib/python3.10/dist-packages/sklearn/linear_model/_logistic.py:458: ConvergenceWarning: lbfgs failed to converge (status=1):
STOP: TOTAL NO. of ITERATIONS REACHED LIMIT.

Increase the number of iterations (max_iter) or scale the data as shown in:
https://scikit-learn.org/stable/modules/preprocessing.html
Please also refer to the documentation for alternative solver options:
https://scikit-learn.org/stable/modules/linear\_model.html#logistic-regression
n_iter_i = _check_optimize_result(
  LogisticRegression()
LogisticRegression()

```

Prediction Output

Source Screenshot Page 8: Evidence extracted from the supplied OptiCrop PDF.

	precision	recall	f1-score	support
apple	1.00	1.00	1.00	4
banana	1.00	1.00	1.00	22
blackgram	0.83	0.99	0.906	21
chickpea	1.00	1.00	1.00	24
coconut	1.00	1.00	1.00	19
coffee	1.00	0.95	0.97	20
cotton	0.91	0.91	0.91	22
grapes	1.00	1.00	1.00	6
jute	0.88	0.82	0.85	17
kidneybeans	0.83	1.00	0.91	20
lentil	0.94	0.94	0.94	17
maize	0.83	0.88	0.86	17
mango	0.95	1.00	0.98	21
mothbeans	0.91	0.81	0.86	26
mungbean	0.92	0.92	0.92	18
muskmelon	1.00	1.00	1.00	14
orange	1.00	1.00	1.00	25
papaya	1.00	0.95	0.98	21
pigeonpeas	1.00	0.78	0.88	18
pomegranate	1.00	1.00	1.00	20
rice	0.89	0.96	0.92	25
watermelon	1.00	1.00	1.00	21
accuracy			0.94	413
macro avg	0.95	0.95	0.95	413
weighted avg	0.95	0.94	0.94	413

```
print("The suggested crop for given climatic condition is: ",prediction)
```

```
/usr/local/lib/python3.10/dist-packages/sklearn/base.py:426: UserWarning: X does not have valid feature names, but LogisticRegression was fitted with feature names
warnings.warn()
```



```

<body>

<div class="topnav">
  <a class="active" href="#home">Home</a>
  <a href="{{ url_for('about') }}">About</a>
  <a href="{{ url_for('findyourcrop') }}">FindYourCrop</a>
</div>

<div style="padding-left:16px">
  <h2>Opti Crop Agricultural Optimisation</h2>
  <p>The main purpose of the OptiCrop aims to develop an advanced software system
</div>

</body>
</html>

```

Prediction Form

```

<body>
<div style="padding-left:16px">
  <h2>Opti Crop Smart Agricultural Optimization Using ML</h2>
  <p>
    The project "Opti Crop: Smart Agricultural Production Optimization Engine" aims to develop an advanced
    software system that utilizes data-driven insights to optimize agricultural production for different crops.
    By integrating key environmental factors such as Nitrogen (N), Phosphorous (P), Potassium (K) levels, soil
    temperature, humidity, pH, rainfall, and crop types, Opti Crop seeks to provide intelligent recommendations
    to farmers for maximizing yields and resource efficiency. The knowledge and insights gained through the
    Opti Crop project can be utilized by agricultural researchers, policymakers, and stakeholders to advance
    the understanding of crop-environment interactions and inform the development of evidence-based agricultural
    strategies. The Opti Crop project revolutionizes agricultural practices by providing farmers with a smart
    production optimization engine. By integrating key environmental factors and crop-specific recommendations,
    Opti Crop enables farmers to achieve optimal yields, improve resource efficiency, and contribute to sustainable
    agricultural practices.
  </p>
</div>
</body>
</html>

```

Source Screenshot Page 10: Evidence extracted from the supplied OptiCrop PDF.

22. Entity Relationship Diagram

OptiCrop can run from CSV files, but the ERD below models a production database design for storing submitted inputs, predictions, crop labels, and model artifacts.

Entity Relationship Diagram

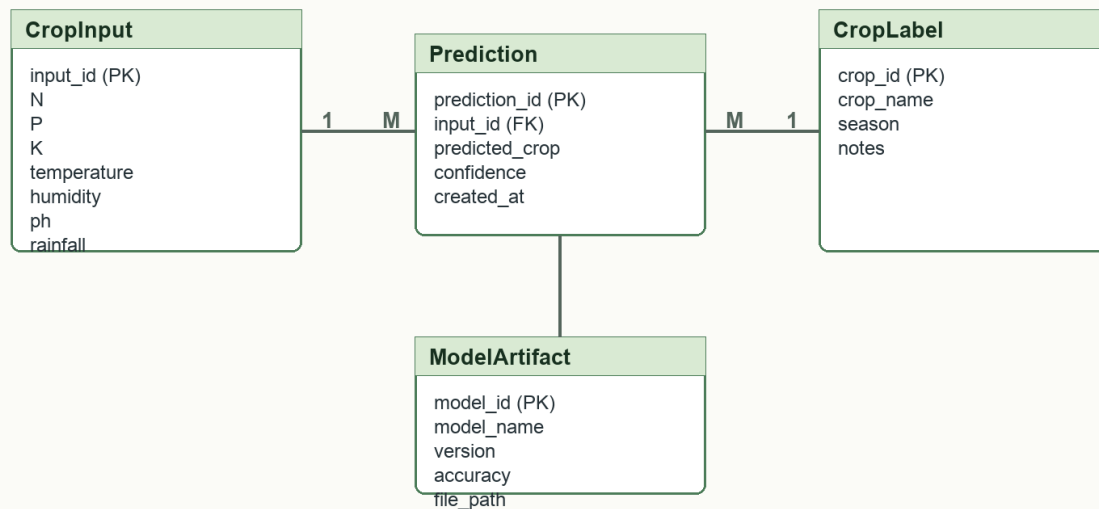


Figure 17: Logical ERD for OptiCrop.

23. Technical Architecture

The architecture separates presentation, application, ML, and data layers. The notebook trains and evaluates models, Pickle stores the selected model, Flask loads model.pkl, and HTML templates collect inputs and display predictions.

OptiCrop Technical Architecture

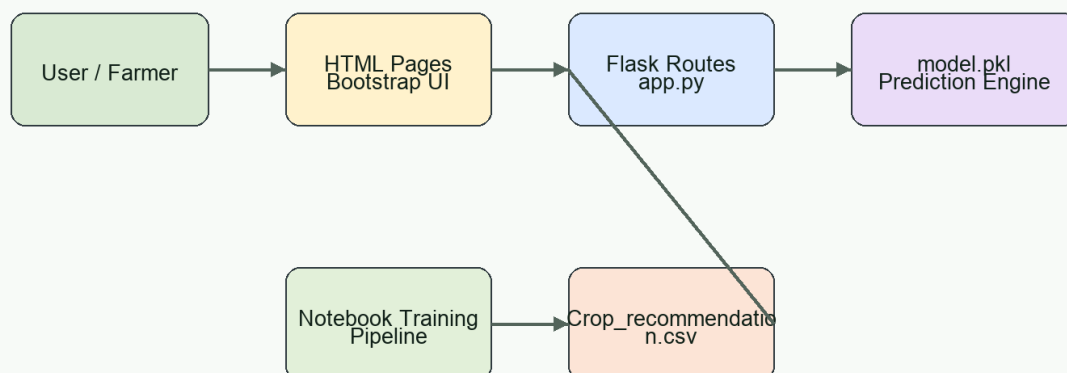


Figure 18: OptiCrop technical architecture.

24. Conclusion

OptiCrop demonstrates a complete end-to-end AI and machine learning workflow for smart agricultural production optimization. It defines a practical problem, uses a structured crop recommendation dataset, performs preprocessing and EDA, trains and compares multiple models, evaluates classification performance, saves the final model, and

deploys prediction through a Flask web application.

The documentation is structured for SkillWallet submission with cover page, requirements, business context, literature survey, dataset description, preprocessing, EDA, seasonal analysis, train-test split, ML models, K-Means elbow method, evaluation, model saving, Flask application details, screenshots, ERD, technical architecture, and conclusion.